

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное
учреждение высшего образования

«Ярославский государственный университет им. П.Г. Демидова»

Кафедра дифференциальных уравнений

Сдано на кафедру

«__» _____ 2026 г.

Заведующий кафедрой

д. ф.-м. н., профессор

_____ Бережной Е. И.

Курсовая работа

Задача классификации приступов эпилепсии

направление подготовки

01.04.02 Прикладная математика и информатика

Научный руководитель

д. ф.-м. н.

_____ Бережной Е. И.

«__» _____ 2026 г.

Студент группы ПМИ-31БО

_____ Вещев Е. Д.

«__» _____ 2026 г.

Ярославль, 2026г.

Содержание

1	Введение	3
2	Теоретические основы	3
2.1	Задача классификации	3
2.2	Недообучение (underfitting)	4
2.3	Переобучение (overfitting)	4
2.4	Дерево решений	4
2.5	Случайный лес (Random Forest)	4
2.6	Градиентный бустинг (Gradient Boosting)	5
2.7	Многослойный перцептрон (MLP)	5
3	Описание датасета BEED	6
4	Экспериментальная часть	7
4.1	Предобработка данных	7
4.2	Обучение классических моделей	7
4.3	Архитектура и обучение MLP	8
4.4	Сравнение глубины сети	9
4.5	Анализ переобучения	9
4.6	Примеры работы модели на тестовых данных	10
5	Выводы	10

1 Введение

Эпилепсия — одно из самых распространённых неврологических заболеваний в мире, для диагностики которого требуется длительный анализ электроэнцефалограмм (ЭЭГ) врачом. Применение методов машинного обучения позволяет автоматизировать процесс распознавания эпилептических приступов и сократить время анализа.

Целью данной работы является построение и сравнение нескольких моделей классификации на открытом датасете ЭЭГ BEED: случайного леса, градиентного бустинга и многослойного перцептрона (MLP).

2 Теоретические основы

2.1 Задача классификации

В машинном обучении классификацией называют задачу отнесения объекта к одному из заранее известных классов на основе его признаков. В данной работе классы соответствуют типам ЭЭГ-активности: здоровые, генерализованные приступы, фокальные приступы и приступные события.

Для оценки качества классификации используются ряд метрик. Основной из них является **accuracy** (точность) — доля правильно предсказанных объектов:

$$\text{accuracy} = \frac{TP + TN}{TP + TN + FP + FN},$$

где

- TP (true positive) — количество объектов, правильно отнесённых к положительному классу (наличие приступа)
- TN (true negative) — количество объектов, правильно отнесённых к отрицательному классу
- FP (false positive) — ошибка первого рода (ложное срабатывание)
- FN (false negative) — ошибка второго рода (пропуск цели)

Кроме того, в процессе обучения нейронных сетей минимизируется функция потерь. Для многоклассовой классификации стандартной является **кросс-энтропия**:

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c}),$$

где

- N — число объектов

- C — количество классов
- $y_{i,c} = 1$, если объект i принадлежит классу c , иначе 0
- $\hat{y}_{i,c}$ — предсказанная моделью вероятность принадлежности к классу c

Также для детального анализа ошибок используется **матрица ошибок**. В случае бинарной классификации она имеет вид:

	Предсказано: 0	Предсказано: 1
Истина: 0	TN	FP
Истина: 1	FN	TP

В нашем случае это матрица, в которой строки соответствуют истинным классам, а столбцы — предсказанным.

2.2 Недообучение (underfitting)

Недообучение – ситуация, когда модель не способна уловить истинную закономерность данных, в силу ее простоты. Недообучение выдает плохую точность, как на обучающей, валидационной так и тестовой выборках.

2.3 Переобучение (overfitting)

Переобучение – ситуация, когда модель запоминает случайные шумы из данных, в силу своей сложности. Переобучение выдает высокую точность на обучающей выборке, но плохую на валидационной и тестовой выборках.

2.4 Дерево решений

Дерево решений – это базовый алгоритм машинного обучения, основанный на логическом разделении признаков. Обыкновенные деревья решений склонны к переобучению, поэтому их обычно используют в ансамблях. В качестве критерия качества разбиения часто используется **критерий Джини**.

$$G = 1 - \sum_{k=1}^K p_k^2,$$

где p_k — доля объектов класса k в узле. Чем меньше значение G , тем «чище» узел. Дерево выбирает разбиение, которое максимизирует снижение неопределённости.

2.5 Случайный лес (Random Forest)

Случайный лес представляет собой ансамбль деревьев решений, каждое из которых строится на случайной подвыборке данных. При построении каждого разбиения дополни-

тельно случайно выбирается подмножество признаков, что снижает корреляцию между деревьями и уменьшает переобучение. Итоговый прогноз — усреднение предсказаний всех деревьев (в случае задачи регрессии), или же голосование (при классификации).

2.6 Градиентный бустинг (Gradient Boosting)

В отличие от случайного леса, градиентный бустинг строит деревья последовательно, где каждое следующее дерево исправляет ошибки предыдущих. Пусть на шаге $m - 1$ построена модель $F_{m-1}(x)$. Тогда очередное дерево $h_m(x)$ обучается аппроксимировать отрицательный градиент функции потерь, а модель обновляется следующим образом:

$$F_m(x) = F_{m-1}(x) + \nu \cdot h_m(x),$$

где ν — скорость обучения (learning rate), контролирующая вклад каждого нового дерева. Процесс повторяется M раз (количество деревьев $n_estimators$). Скорость обучения (learning rate) контролирует вклад каждого нового дерева. Стохастический градиентный бустинг ($subsample < 1$) использует случайную подвыборку объектов на каждой итерации, что улучшает обобщение.

2.7 Многослойный перцептрон (MLP)

MLP — это полносвязная нейронная сеть, состоящая из входного слоя (размерность признаков), нескольких скрытых слоёв и выходного слоя (число классов). В качестве функций активации использовались ReLU и GELU:

$$\text{ReLU}(x) = \max(0, x)$$

$$\text{GELU}(x) \approx 0.5 x \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + 0.044715 x^3) \right) \right)$$

Для борьбы с переобучением применялись:

- **Batch Normalization**

Batch Normalization — метод, который нормализует активации внутри сети для каждого мини-батча, ускоряя обучение и повышая его стабильность.

Пусть для мини-батча $\mathcal{B} = \{x_1, \dots, x_m\}$ заданы активации x_i . Тогда операции BatchNorm определяются следующим образом:

$$\begin{aligned} \mu_{\mathcal{B}} &= \frac{1}{m} \sum_{i=1}^m x_i, & \sigma_{\mathcal{B}}^2 &= \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2, \\ \hat{x}_i &= \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}, & y_i &= \gamma \hat{x}_i + \beta, \end{aligned}$$

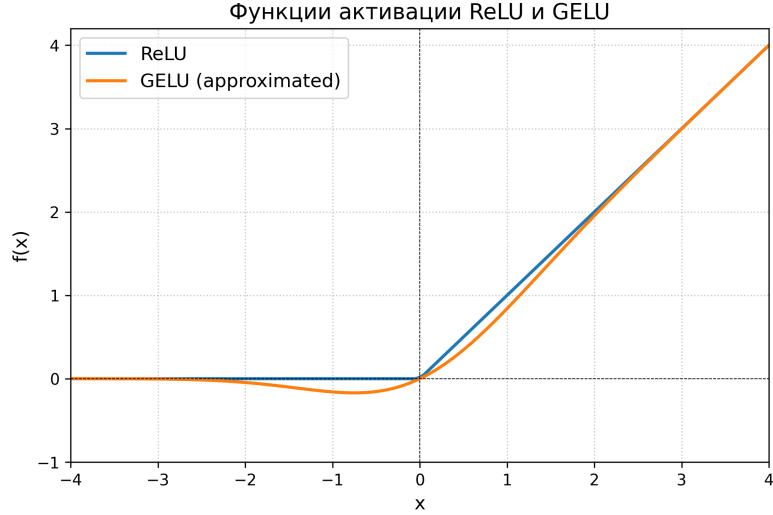


Рис. 1: Сравнение функций активации ReLU и GELU. GELU плавно приглушает отрицательные значения, в отличие от ReLU, который полностью их обнуляет.

где:

- μ_B — среднее значение по батчу;
- σ_B^2 — дисперсия по батчу;
- ϵ — малая константа ($\approx 10^{-5}$), добавляемая для численной устойчивости;
- $\hat{x}_i$ — нормализованное значение;
- γ, β — обучаемые параметры масштаба и сдвига, позволяющие сети при необходимости восстанавливать исходное распределение активаций.

После вычисления y_i результат передаётся в функцию активации (например, ReLU или GELU).

- **Dropout** — случайное отключение нейронов для повышения устойчивости.

3 Описание датасета BEED

BEED (Bangalore EEG Epilepsy Dataset) содержит 8000 записей многоканальной ЭЭГ. Количество каналов — 16. Все образцы размечены на 4 класса (по 2000 в каждом):

- 0 — здоровые испытуемые;
- 1 — генерализованные приступы;
- 2 — фокальные приступы;
- 3 — приступные события.

Данные представлены в табличном виде (CSV), где каждая строка — это набор из 16 числовых признаков, соответствующих усреднённой активности каналов. Временная структура внутри фрагмента отсутствует, поэтому для анализа применимы только классические методы машинного обучения и MLP, но не свёрточные сети по времени.

4 Экспериментальная часть

4.1 Предобработка данных

Для ансамблевых моделей выборка разделена на три части: тренировочная (60%), валидационная (20%) и тестовая (20%) со стратификацией по классам. Пример кода предобработки приведён в листинге 1.

```
1 beed_file_path = 'BEED_Data.csv'
2 beed_data = pd.read_csv(beed_file_path)
3
4 X = beed_data.iloc[:, :-1]
5 y = beed_data.iloc[:, -1]
6
7
8 def preprocess_data():
9     global X, y
10    X_temp, X_test, y_temp, y_test = train_test_split(
11        X, y, test_size=0.2, random_state=1, stratify=y
12    )
13    sss = StratifiedShuffleSplit(n_splits=10, test_size=0.25,
14                                random_state=1)
15    return sss, X_temp, X_test, y_temp, y_test
```

Листинг 1: Функция предобработки (final_models.py)

4.2 Обучение классических моделей

Случайный лес и градиентный бустинг реализованы с помощью библиотеки `scikit-learn`. Гиперпараметры подбирались по валидационной выборке с помощью `GridSearchCV`. Лучшие конфигурации:

- **Random Forest:** `n_estimators=400`, `min_samples_leaf=1`, `min_samples_split=2`,
- **Gradient Boosting:** `n_estimators=500`, `learning_rate=0.1`, `max_depth=12`, `subsample=0.7`.

Результаты на тестовой выборке сведены в таблицу 1.

Таблица 1: Точность классических моделей

Модель	Accuracy (test)	Время обучения (с)
Random Forest	96.56%	19.4
Gradient Boosting	97.31%	2.48

4.3 Архитектура и обучение MLP

Была исследована глубокая полносвязная сеть с 7 скрытыми слоями (архитектура: $16 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 4$). Код модели приведён в листинге 2.

```

1
2 class Net(nn.Module):
3     def __init__(self, input_size=16, num_classes=4):
4         super().__init__()
5         self.fc1 = nn.Linear(input_size, 128)
6         self.bn1 = nn.BatchNorm1d(128)
7         self.fc2 = nn.Linear(128, 256)
8         self.bn2 = nn.BatchNorm1d(256)
9         self.fc3 = nn.Linear(256, 512)
10        self.bn3 = nn.BatchNorm1d(512)
11        self.fc4 = nn.Linear(512, 256)
12        self.bn4 = nn.BatchNorm1d(256)
13        self.fc5 = nn.Linear(256, 128)
14        self.bn5 = nn.BatchNorm1d(128)
15        self.fc6 = nn.Linear(128, 64)
16        self.bn6 = nn.BatchNorm1d(64)
17        self.fc7 = nn.Linear(64, num_classes)
18        self.dropout = nn.Dropout(0.5)
19
20    def forward(self, x):
21        x = F.gelu(self.bn1(self.fc1(x)))
22        x = self.dropout(x)
23        x = F.gelu(self.bn2(self.fc2(x)))
24        x = F.gelu(self.bn3(self.fc3(x)))
25        x = F.gelu(self.bn4(self.fc4(x)))
26        x = F.gelu(self.bn5(self.fc5(x)))
27        x = F.gelu(self.bn6(self.fc6(x)))
28        x = self.fc7(x)
29        return x

```

Листинг 2: Класс MLP с 7 слоями (beed_network.py)

Обучение производилось в течение 150 эпох с размером батча 256, для обновления весов сети использовался стохастический градиентный спуск с моментом. Правила

обновления имеют вид:

$$v_{t+1} = \mu v_t - \eta \nabla L(w_t), \quad w_{t+1} = w_t + v_{t+1},$$

где $\mu = 0.9$ — коэффициент момента, $\eta = 0.01$ — скорость обучения, $\nabla L(w_t)$ — градиент функции потерь по весам на шаге t . Момент позволяет ускорить сходимость и сгладить колебания градиента. Функция потерь — CrossEntropyLoss.

4.4 Сравнение глубины сети

Для демонстрации влияния глубины были обучены также сети с 3 и 15 скрытыми слоями. Результаты представлены в таблице 2.

Таблица 2: Влияние числа слоёв на точность		
Число слоёв	Точность на тесте	Примечание
3	95.6%	Быстрое обучение
7	98.75%	Оптимальная глубина
15	82.4%	Переобучение

4.5 Анализ переобучения

На рисунке 2 показаны кривые тренировочного и валидационного loss для 7-слойной сети. Видно, что оба loss синхронно снижаются и не расходятся, что свидетельствует об отсутствии переобучения. Также для предотвращения переобучения могла использоваться ранняя остановка (обучение прекращается в случае, когда валидационная ошибка не уменьшается в течение заданного числа эпох)

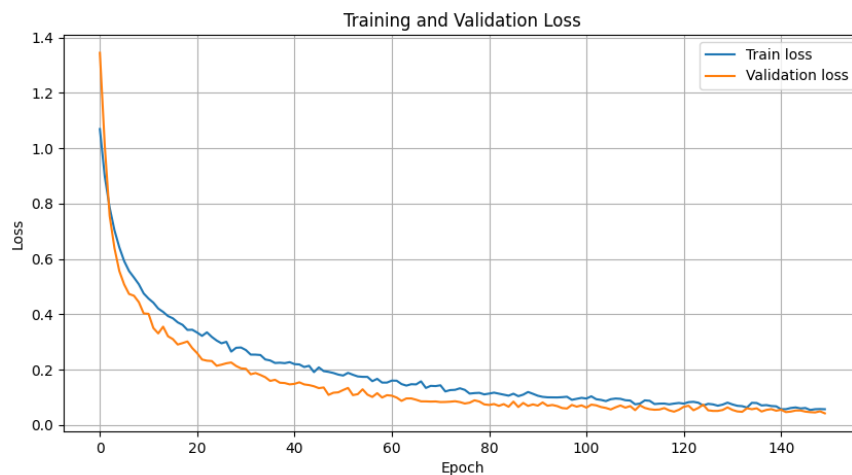


Рис. 2: График изменения train и val loss

Дополнительная проверка на разных случайных разбиениях дала среднюю точность $98.7\% \pm 0.3\%$, что подтверждает стабильность модели.

4.6 Примеры работы модели на тестовых данных

Для иллюстрации работы обученной MLP (7 слоёв) приведём конкретный пример из тестовой выборки.

Таблица 3: Пример ошибочного предсказания MLP

Параметр	Значение
Истинный класс	2 (фокальный приступ)
Предсказанный класс	3 (приступное событие)
Признаки (каналы X1–X16)	
X1	0.0405
X2	-0.0501
X3	0.2860
X4	0.2789
X5	0.0750
X6	-0.0190
X7	0.1210
X8	0.3683
X9	-0.1929
X10	0.3879
X11	0.5496
X12	0.4212
X13	-0.2841
X14	0.6526
X15	0.6720
X16	0.1980

Как видно из таблицы, значения признаков данного образца (например, высокие положительные значения X14, X15, X11) могут быть характерны и для класса 3, что привело к неверной классификации. Подобные ошибки встречаются редко (менее 2% от общего числа тестовых примеров) и в основном сосредоточены на различении классов 1–3, что согласуется с матрицей ошибок.

5 Выводы

В ходе работы были решены следующие задачи:

1. Проведён анализ датасета BEED, выполнена предобработка.
2. Обучены и сравнены Random Forest (96.56%), Gradient Boosting (97.31%) и MLP.
3. Подобрана оптимальная архитектура MLP (7 слоёв), достигнута точность 98.75% на тестовой выборке.
4. Исследовано влияние глубины сети: увеличение до 15 слоёв приводит к переобучению (82.4%).

5. Показано, что модель не переобучена (кривые loss сходятся, стабильность при разных разбиениях).

Практическая значимость работы заключается в демонстрации возможности автоматической классификации ЭЭГ с очень высокой точностью, что может быть использовано для поддержки принятия решений в клинической практике. Исходные коды всех экспериментов, включая обучение Random Forest, XGBoost и MLP с 7 слоями, а также скрипты для визуализации и расчёта метрик, опубликованы на GitHub: https://github.com/Gulaal/epilepsy_dataset_ml

Список литературы

- [1] Иоффе С., Шегеди К. *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift* // Proceedings of the 32nd International Conference on Machine Learning (ICML). — 2015. — С. 448–456. arXiv:1502.03167.
- [2] Хендрикс Д., Гимпел К. *Gaussian Error Linear Units (GELUs)* // arXiv preprint arXiv:1606.08415. — 2016.
- [3] Брейман Л. *Random Forests* // Machine Learning. — 2001. — Т. 45, № 1. — С. 5–32.
- [4] Чэнь Т., Гестрин К. *XGBoost: A Scalable Tree Boosting System* // Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. — 2016. — С. 785–794.
- [5] Фридман Дж. Х. *Greedy Function Approximation: A Gradient Boosting Machine* // The Annals of Statistics. — 2001. — Т. 29, № 5. — С. 1189–1232.
- [6] Кингма Д. П., Ба Дж. Л. *Adam: A Method for Stochastic Optimization* // arXiv preprint arXiv:1412.6980. — 2014.
- [7] *BEED: Bangalore EEG Epilepsy Dataset* // UCI Machine Learning Repository. — 2025. — Режим доступа: <https://archive.ics.uci.edu/dataset/1134/beed>
- [8] *PyTorch Documentation: torch.nn.BatchNorm1d* // PyTorch. — Режим доступа: <https://pytorch.org/docs/stable/generated/torch.nn.BatchNorm1d.html>
- [9] *Scikit-learn Documentation: RandomForestClassifier* // Scikit-learn. — Режим доступа: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>
- [10] *XGBoost Documentation: Python API Reference* // XGBoost. — Режим доступа: https://xgboost.readthedocs.io/en/stable/python/python_api.html